

Towards Dev · [Follow publication](#)

Atomic Operations, Race Conditions, and Critical Sections — A Clear Introduction

6 min read · Mar 18, 2026



Sagar

Following ▾

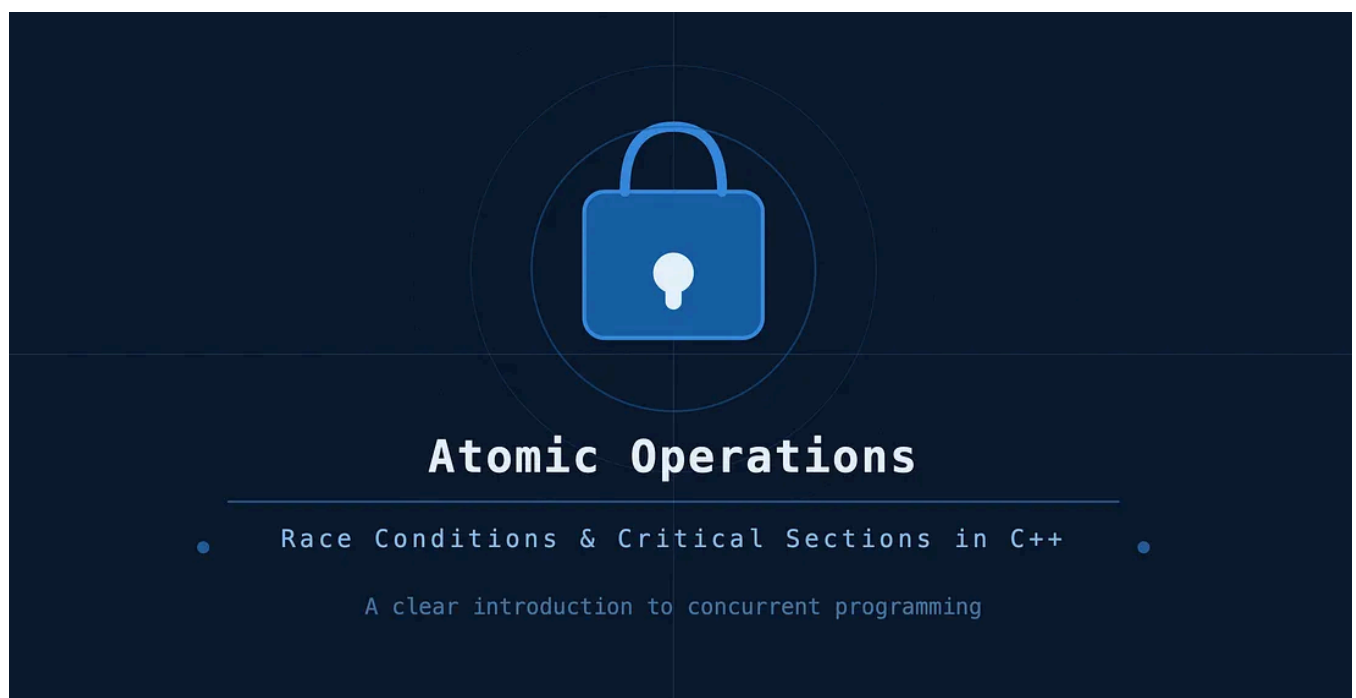


Listen



Share

... More



Before we dive in, let's set expectations !

This article is focused on building a strong intuition around **race conditions, atomic operations, and critical sections** — the *why* behind multithreading bugs.

We will not go deep into synchronization mechanisms like `std::atomic`, mutexes, or memory ordering just yet.

Instead, the goal is to understand:

- Why seemingly simple code breaks under concurrency
- What “atomic” really means at a low level
- How race conditions actually happen in practice

In upcoming articles, we’ll dive into **how to fix these problems properly** using `std::atomic`, locks, and modern C++ synchronization primitives.

So, What’s the Big Deal with `x = x + 1` ?

Alright, let’s start with something that *looks* dead simple:

```
// simple_increment.cpp
// Compile: g++ -std=c++17 -o simple_increment simple_increment.cpp

#include <iostream>

int main() {
    int x = 5;

    // Looks like one operation, right?
    // Spoiler: it's not.
    x = x + 1;

    std::cout << "x = " << x << "\n";    // x = 6
    return 0;
}
```

```
$ ./simple_increment
x = 6
```

One line. One variable. One result. Nothing can go wrong here... right?

Well, in a single-threaded world — sure. But the moment we add a second thread, this innocent little line becomes a ticking time bomb. And to understand why, we need to see what the CPU *actually does* when it runs `x = x + 1`.

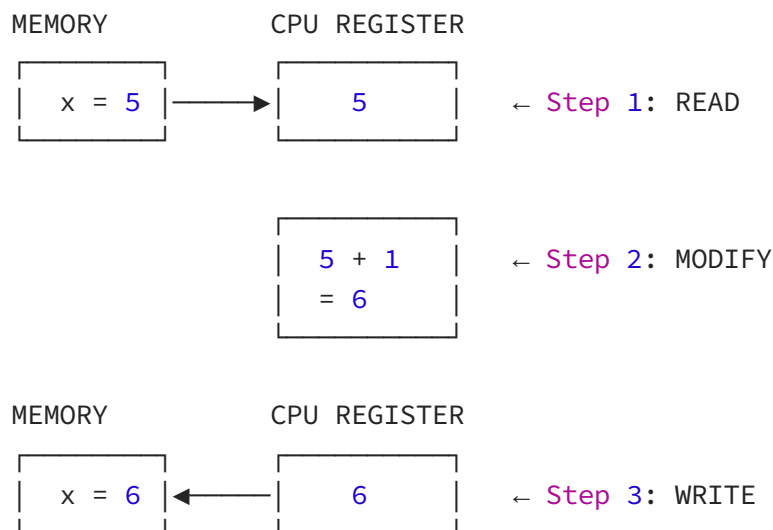
What Really Happens Under the Hood ?

See, when you write $x = x + 1$, you think of it as one action. But your CPU? It doesn't see one action. It sees three distinct steps:

```
Step 1: READ   - Load the value of x from memory into a CPU register
Step 2: MODIFY - Increment the value inside the register (register = register + 1)
Step 3: WRITE  - Store the new value from the register back to memory
```

This is commonly called the **Read-Modify-Write** cycle.

Let me draw it out for you:



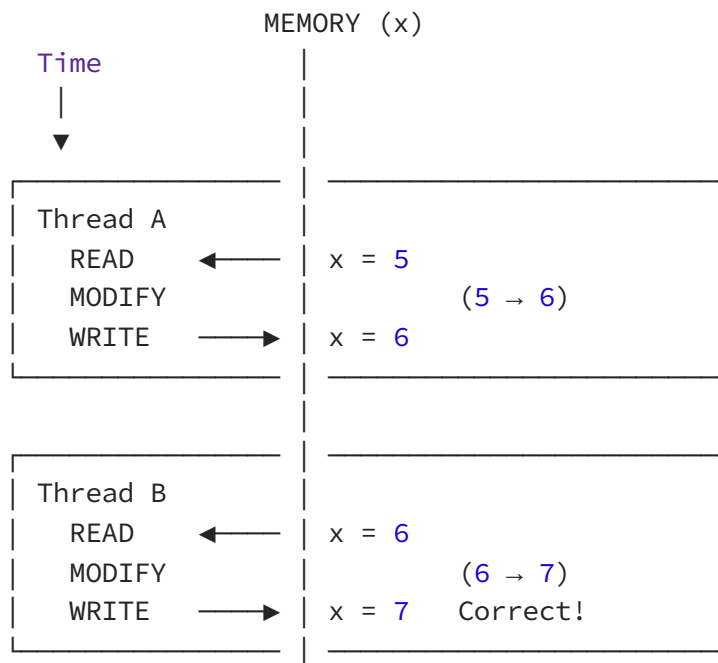
Simple enough when one thread is doing this. The trouble starts when you have two or more threads trying to do the same thing at the same time.

Where Things Go Horribly Wrong ?

Imagine two threads, Thread A and Thread B, both trying to do $x = x + 1$ on a shared variable where x starts at 5.

You'd expect the final answer to be 7 (incremented twice), right?

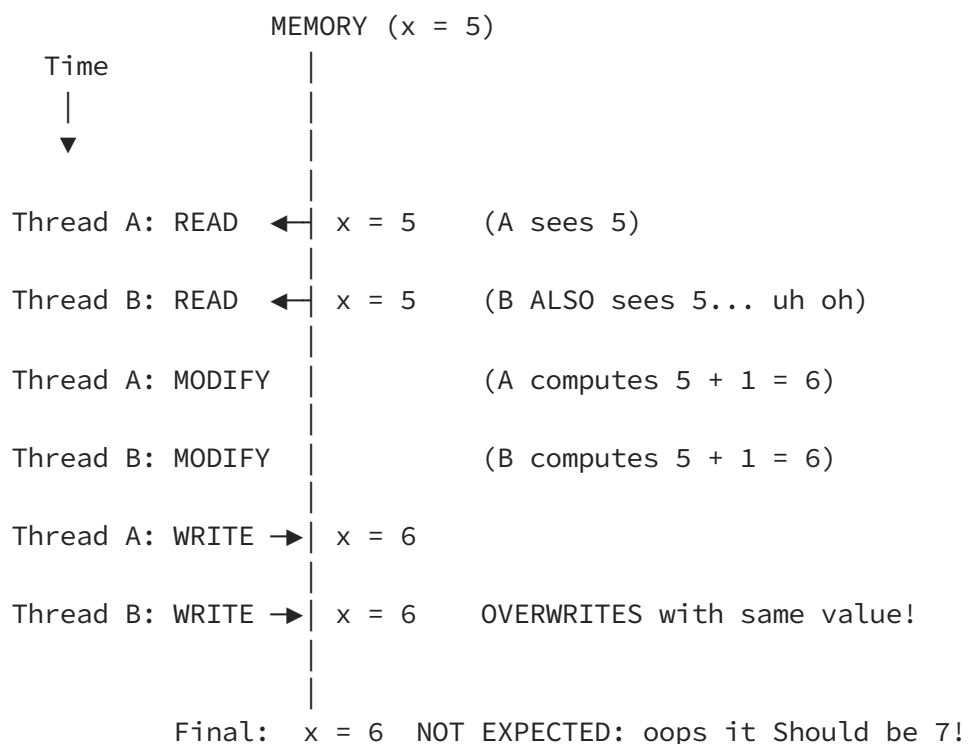
So the **happy path** would look something like this:



Great. $x = 7$. Everybody's happy.

The Nightmare Path (What Can Actually Happen) — Race Condition:

But threads don't politely wait for each other. They run *concurrently*. So this can happen:

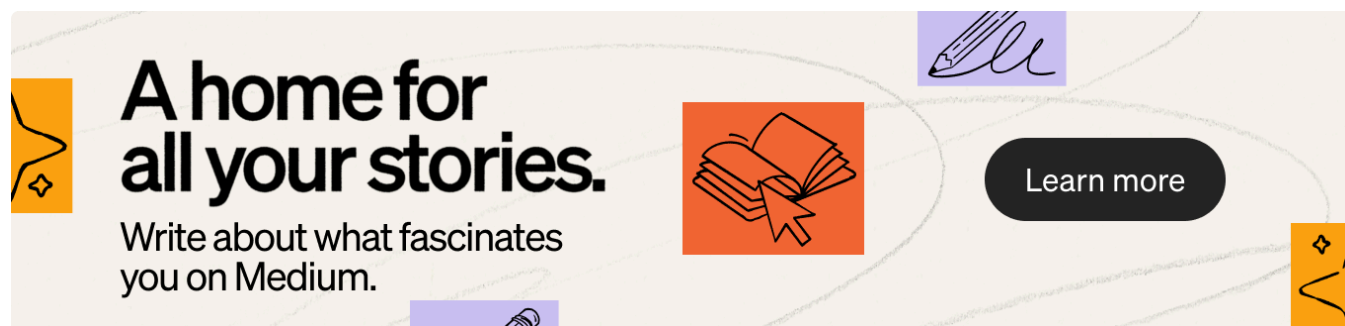


We lost an increment. Thread B read the *stale* value before Thread A could finish writing. Both threads thought `x` was 5, both wrote 6, and one update just... vanished.

This, my friends, is called a **race condition**. And it's one of the nastiest bugs you'll ever chase because it doesn't happen every time — only when the timing is *just right* (or just wrong, depending on how you look at it). the behavior of the program depends on which thread “wins the race” at each step.

So What Is an Atomic Operation ?

We need a way to make those three steps — READ, MODIFY, WRITE — happen as one unbreakable unit. No other thread can sneak in between the steps.



An **atomic operation** is one that executes as a single, indivisible unit. No other thread can observe it in a half-finished state. It either hasn't started, or it's completely done — there's no in-between.

Atomic comes from the Greek word atomos, meaning “indivisible”. An atomic operation is something that either happens completely or not at all — nothing can interrupt it halfway through.

Think of it like a transaction at a bank. Either all the steps complete, or none of them do. Nobody can peek inside mid-transaction and see a corrupted state.

NON-**ATOMIC** $x = x + 1$

READ ← Thread B can

ATOMIC $x = x + 1$

┌ READ

└

MODIFY sneak in here
WRITE

| MODIFY ← Thread B is LOCKED
└ WRITE

| ← all 3 steps
└ as ONE unit

When Thread A performs an atomic increment, Thread B **has to wait**. It cannot interleave. This guarantees correctness.

Note:

atomic operations don't necessarily use locks — they are typically implemented using special CPU instructions that guarantee indivisibility without blocking other threads.

One more thing worth mentioning is **Critical Section**:

*A **critical section** is a piece of code that accesses shared data and **must not be executed by more than one thread at the same time**, because doing so could lead to incorrect results.*

In simple terms, whenever multiple threads operate on the same shared data concurrently, that code becomes a critical section if concurrent execution can break correctness.

*If a critical section is not properly protected (for example, using atomics or synchronization mechanisms), it can lead to **race conditions**.*

Real C++ Code — The Problem in Action

Let's see the race condition happen in actual code:

```
#include <iostream>
#include <thread>

int counter = 0; // shared variable

void increment() {
    for (int i = 0; i < 100000; i++) {
        counter = counter + 1; // NOT atomic — 3 steps under the hood
    }
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
}
```

```
t1.join();
t2.join();

// Expected: 200000
// Actual: could be anything less – e.g., 143821, 178432, etc.
std::cout << "Counter: " << counter << std::endl;

return 0;
}
```

Run this multiple times and you'll get a different answer each time. That's the race condition in all its ugly glory.

Preventing Race Conditions

So far, we've seen how race conditions occur and why atomicity matters. But how do we actually prevent them in real-world applications?

There are several ways to protect critical sections and ensure correct behavior in multithreaded programs:

- **Mutexes (Locks)** — Ensure that only one thread can execute a critical section at a time
- **Semaphores** — Control access to shared resources with more flexible coordination
- **Atomic Operations (`std::atomic`)** — Provide lock-free, thread-safe operations for simple shared data
- **Condition Variables** — Allow threads to wait and coordinate based on certain conditions
- **Latches (C++20)** — One-time synchronization points where threads wait until a counter reaches zero
- **Barriers (C++20)** — Reusable synchronization points where a group of threads must all reach before continuing

While mutexes and atomics protect data, constructs like latches and barriers are primarily used to coordinate execution flow between threads.

Each of these tools comes with its own trade-offs in terms of performance, complexity, and use cases.

Choosing the right synchronization mechanism depends on what you're trying to protect and how your threads interact.

What's Next ?

In this article, i mostly focused on:

- Understanding race conditions
- Breaking down atomic operations
- Identifying critical sections

In the next articles of this series, we'll explore how to actually fix these problems using `std::atomic`, mutexes, and modern C++ synchronization tools.

Concurrency

Programming

Multithreading

Cpp

C Plus Plus Language



Follow

Published in Towards Dev

10K followers · Last published 1 day ago

A publication for sharing projects, ideas, codes, and new theories.